Machine Learning and Data Mining

Decision Trees

Xiuxia Du, Ph.D.
Department of Bioinformatics and Genomics
University of North Carolina at Charlotte

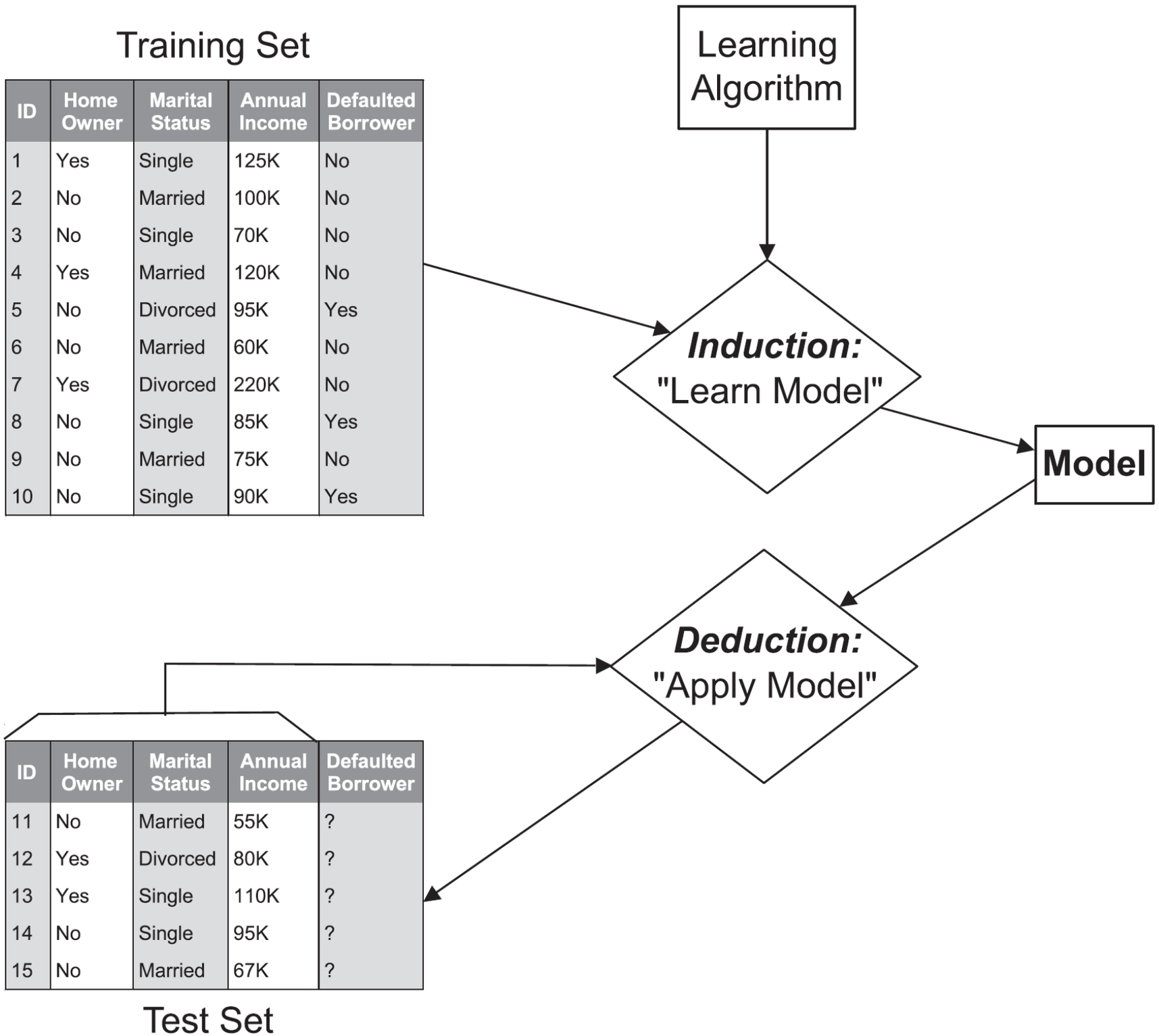
Learning Objectives

- Understand decision trees
- How trees are built?
- Classification vs regression
- Overfitting and pruning
- Python implementation

Some of the materials here are from https://www-users.cse.umn.edu/~kumar001/dmbook/index.php

Understanding Decision Trees

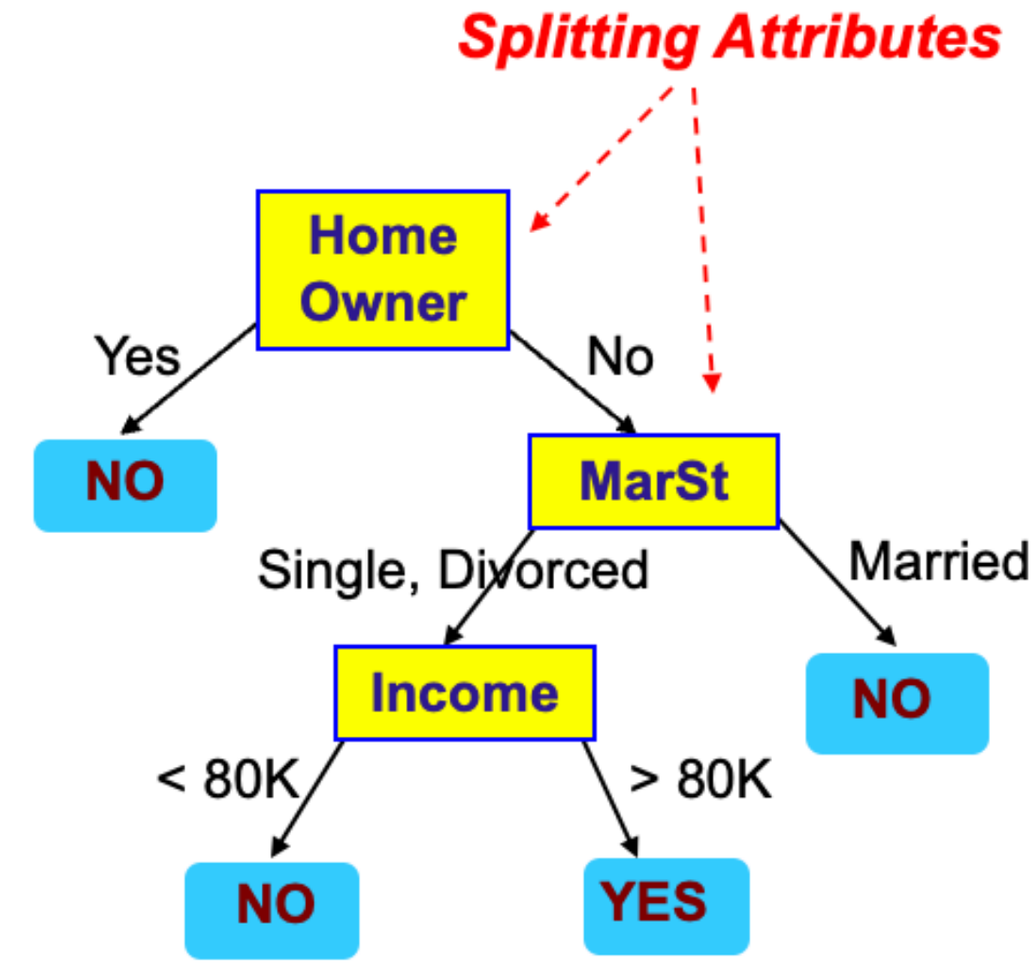
Building and Using a Predictive Model



Example of a Decision Tree

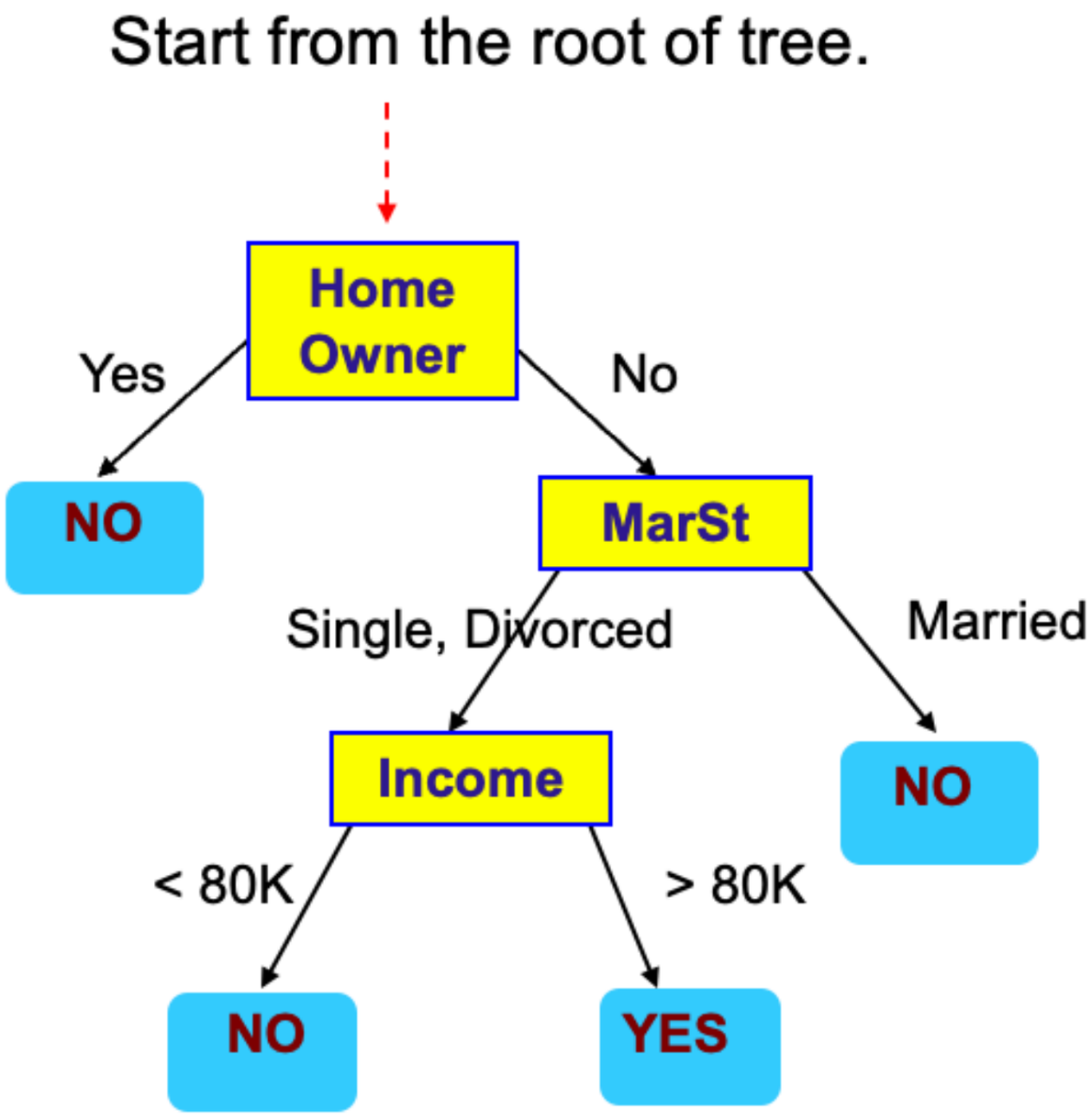
ID	Home Owner (categorical)	Marital Status (categorical)	Annual Income (continuous)	Defaulted Borrower (class)
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data



Model: Decision Tree

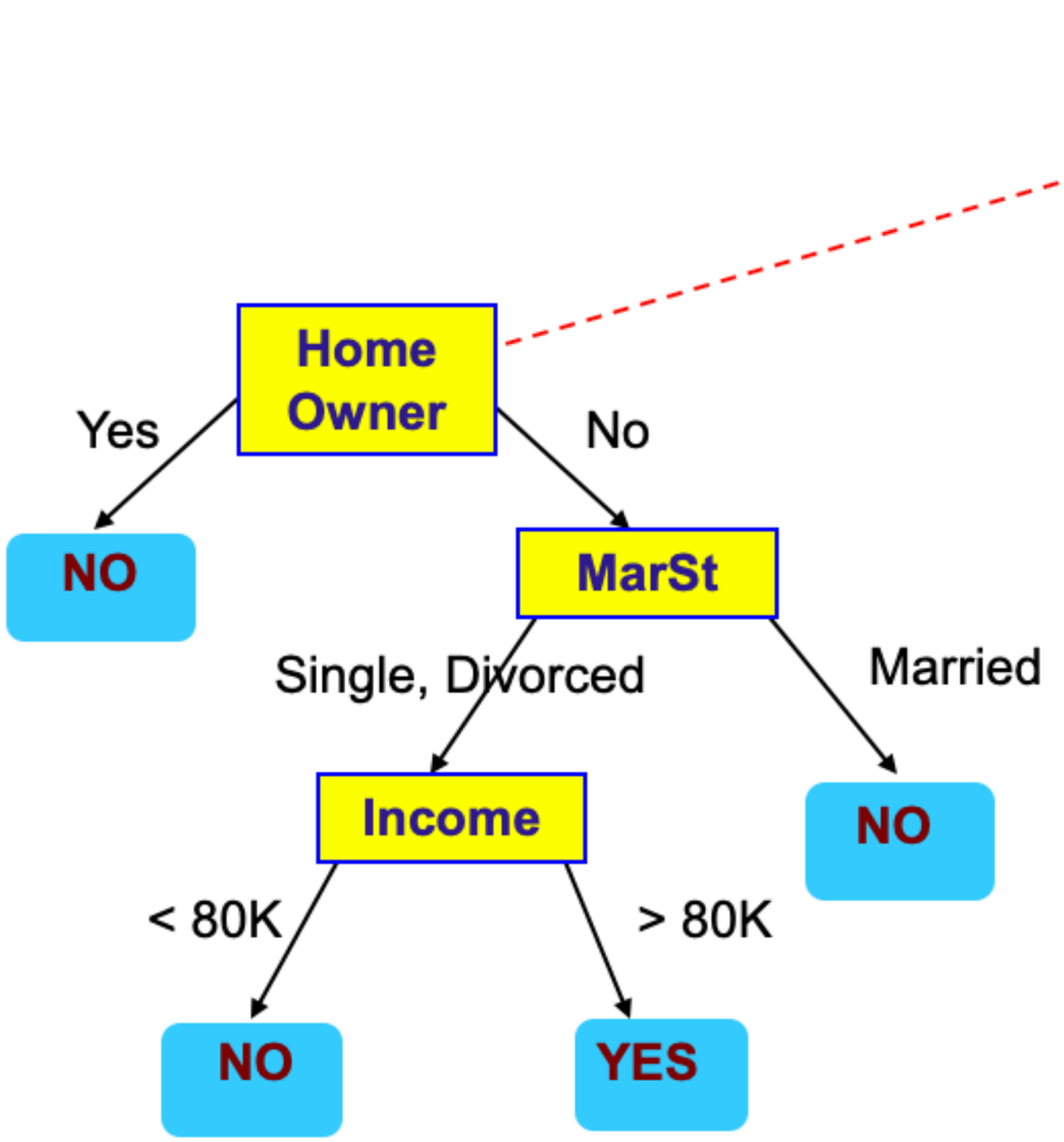
Apply Model to Test Data



Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Apply Model to Test Data



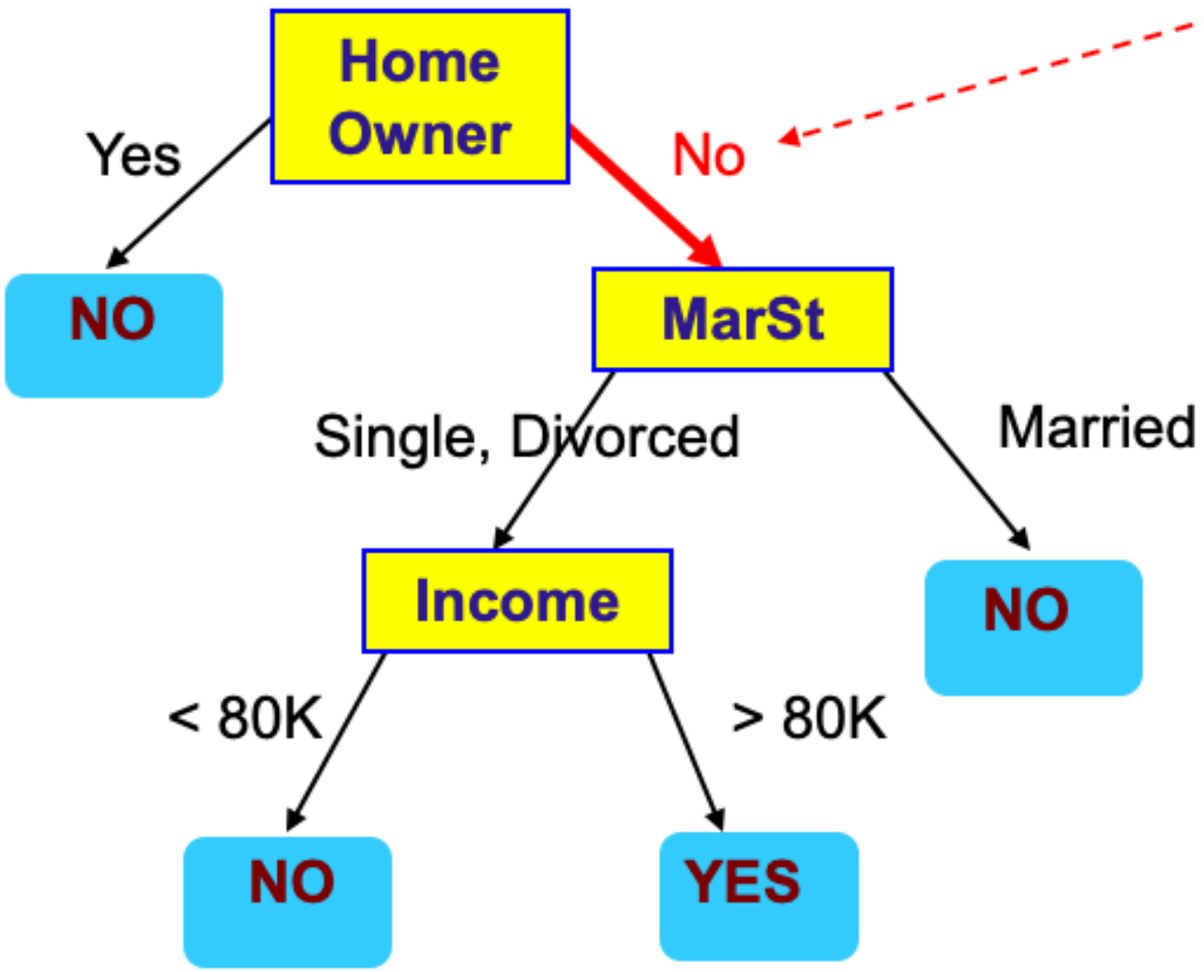
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?

Apply Model to Test Data

Test Data

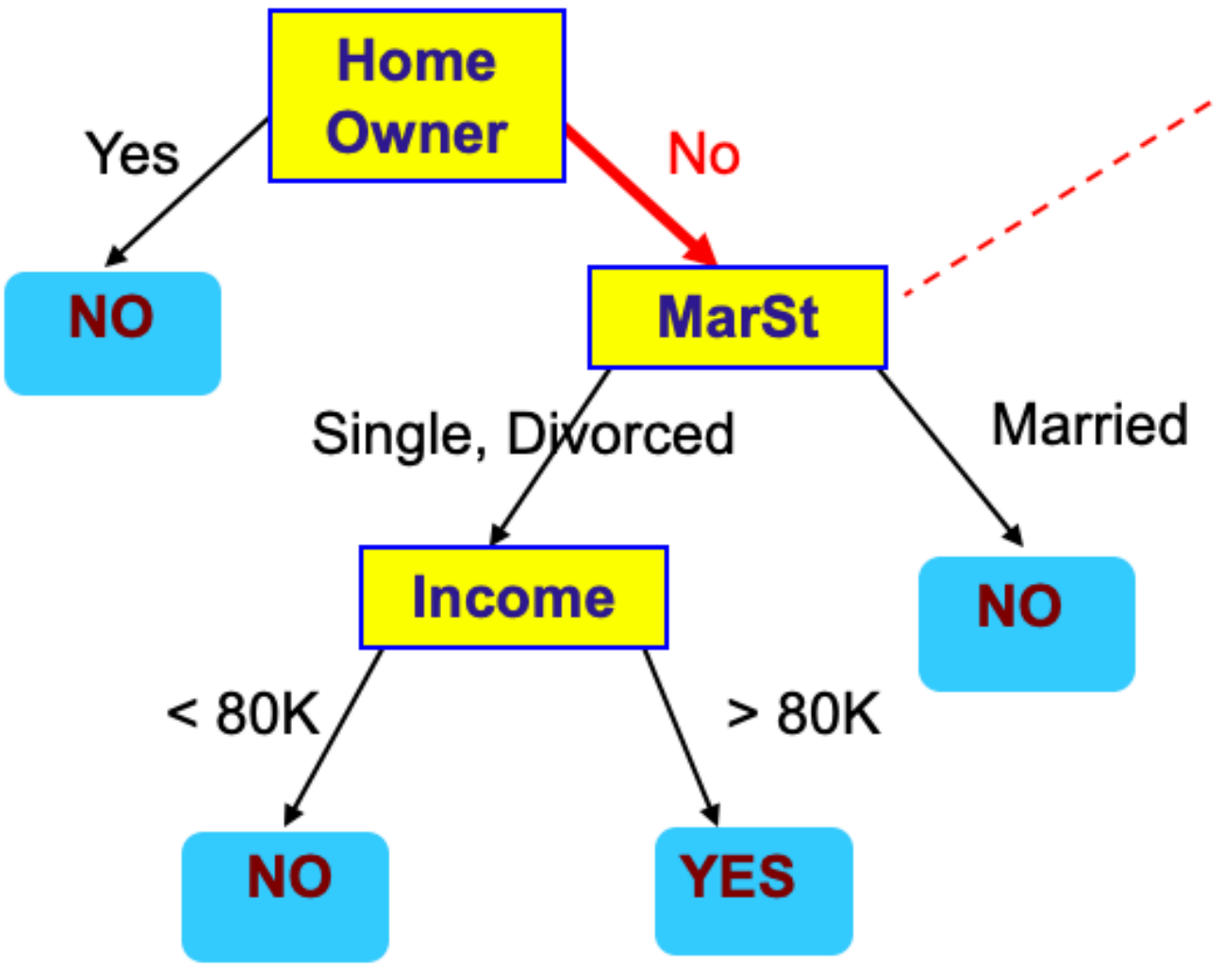
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

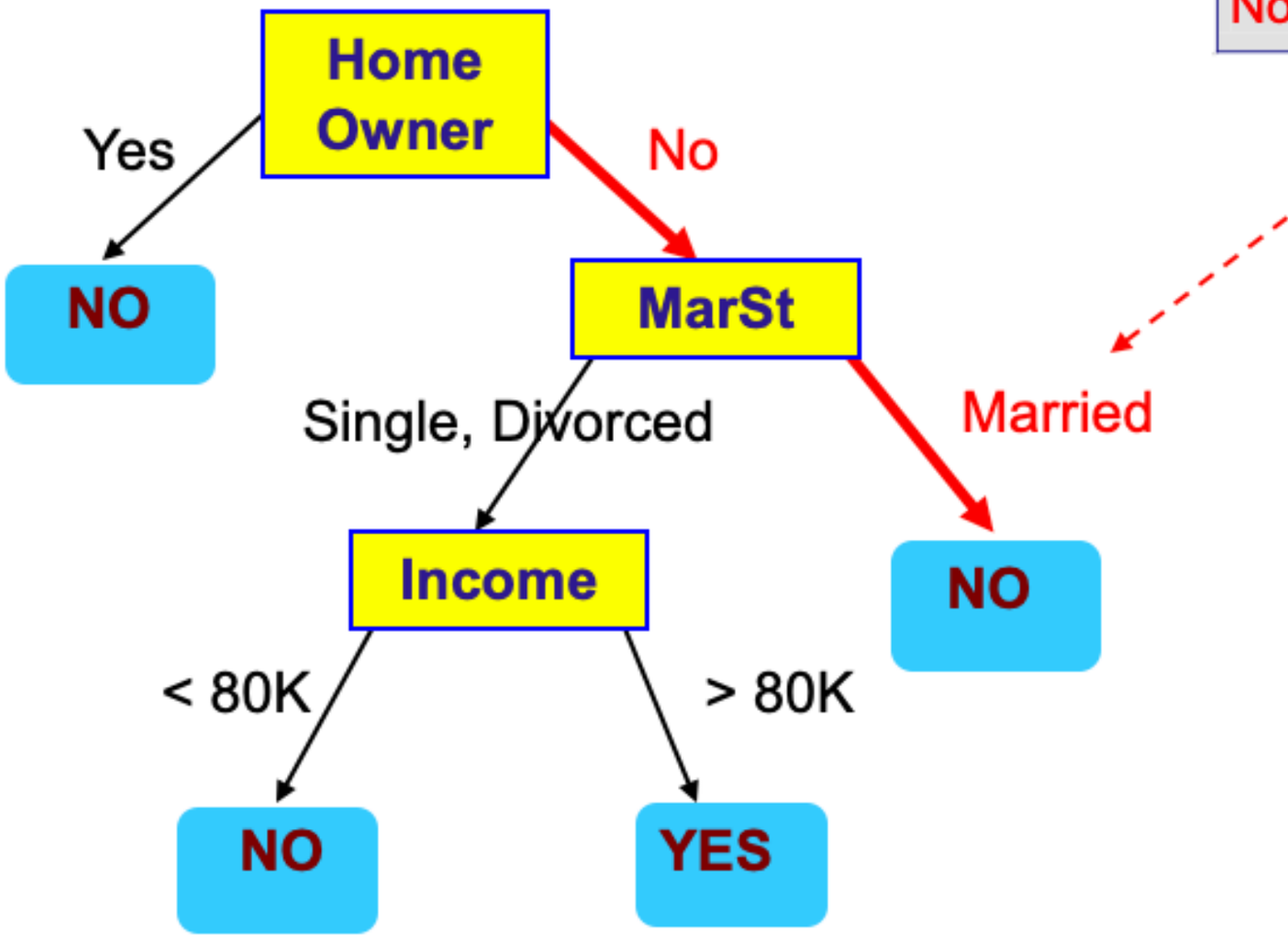
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

Test Data

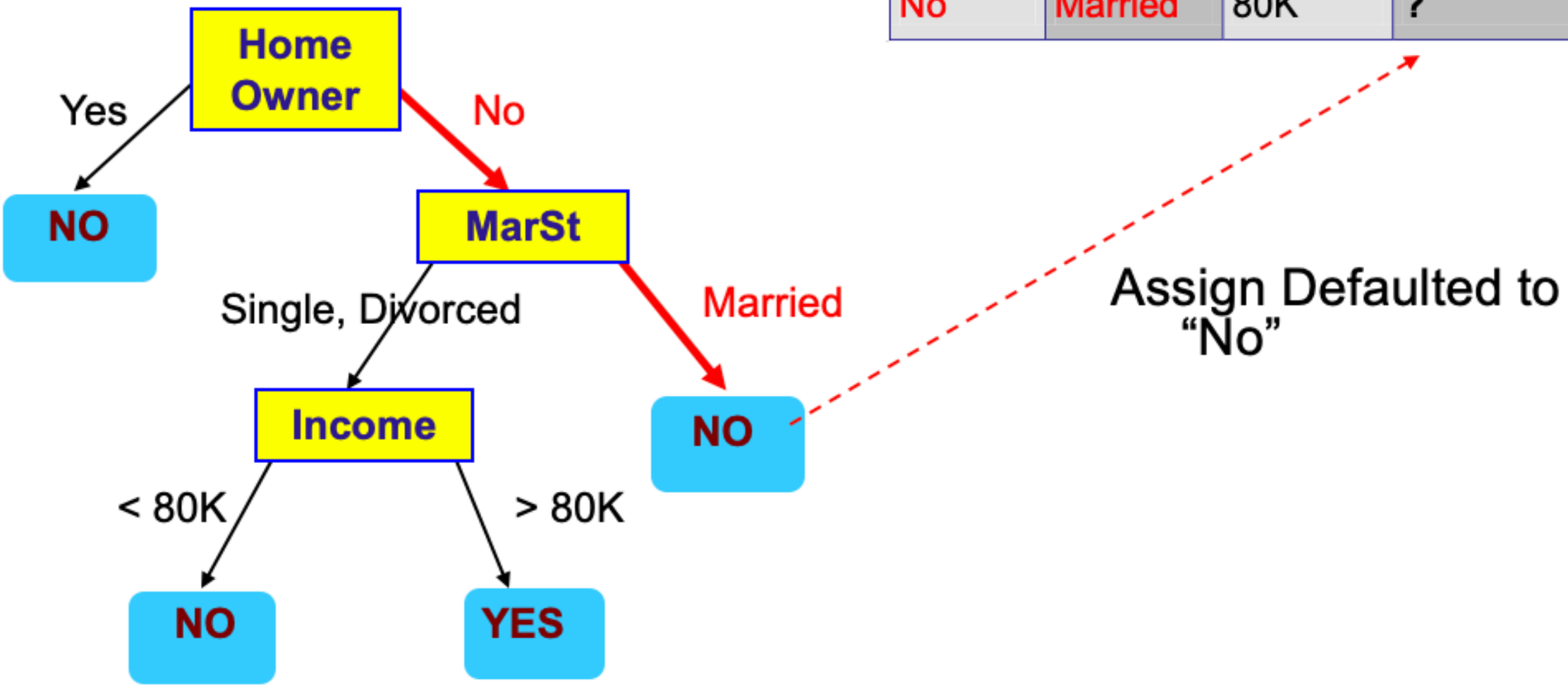
Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Apply Model to Test Data

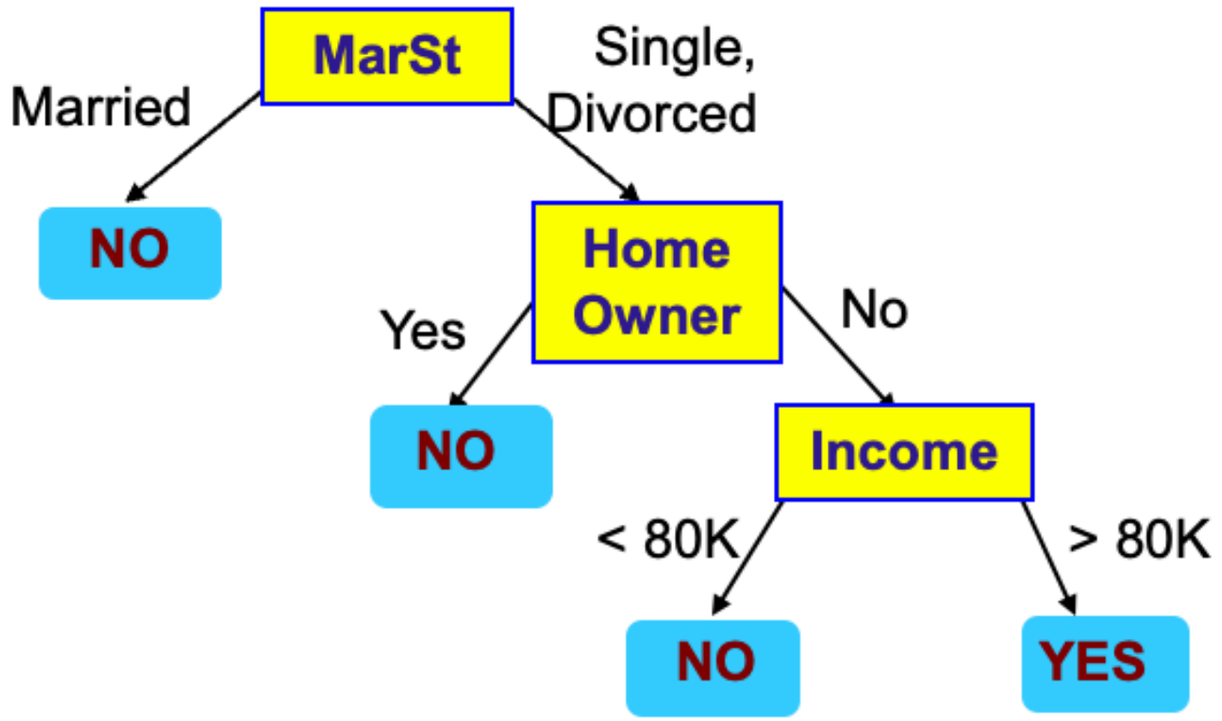
Test Data

Home Owner	Marital Status	Annual Income	Defaulted Borrower
No	Married	80K	?



Another Example of a Decision Tree

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
	categorical	categorical	continuous	class
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

How Decision Trees Are Built?

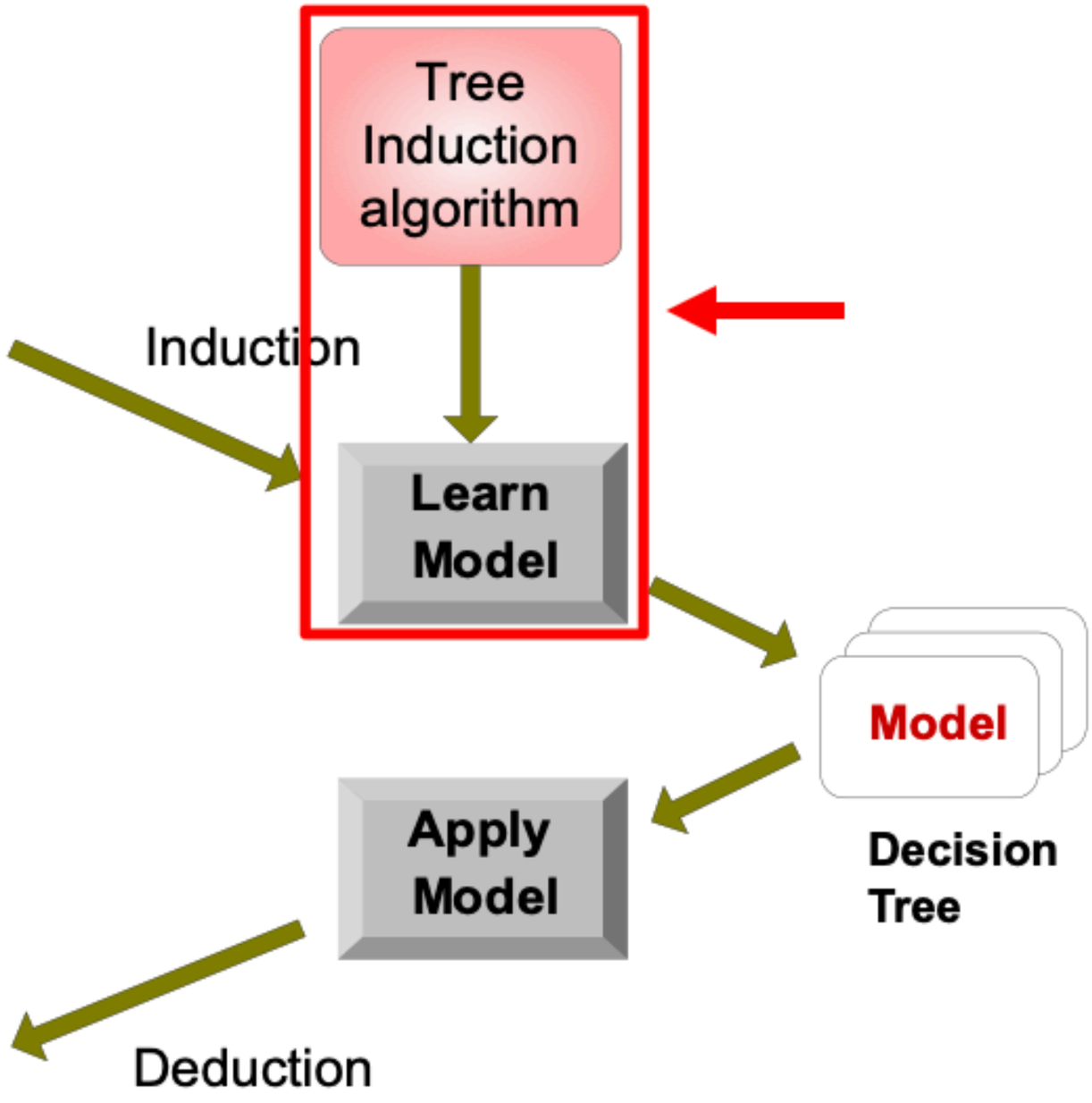
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set

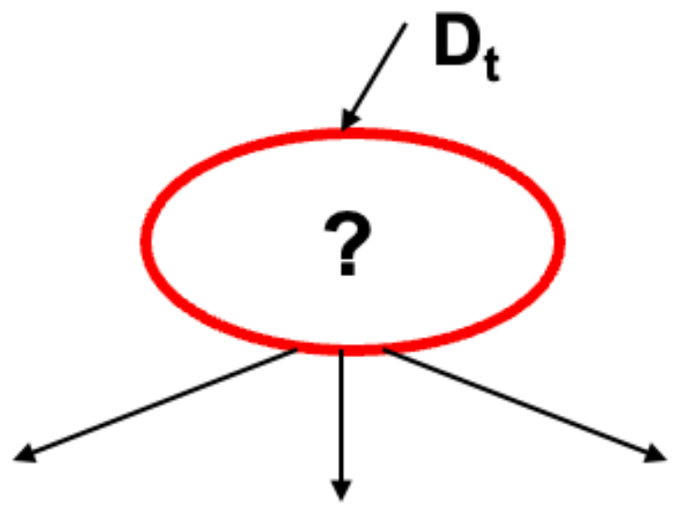


Decision Tree Induction (Hunt's Algorithm)

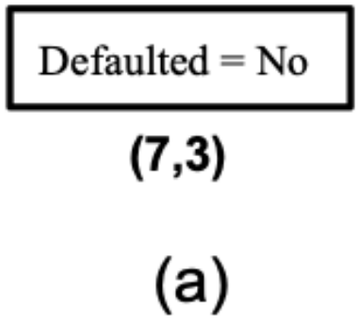
- Let D_t be the set of training records that reach a node t

- General Procedure:
  - If D_t contains records that belong the same class y_t, then t is a leaf node labeled as y_t
  - If D_t contains records that belong to more than one class, use an attribute test to split the data into smaller subsets. Recursively apply the procedure to each subset.

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

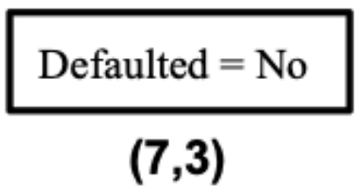


Decision Tree Induction (Hunt's Algorithm)

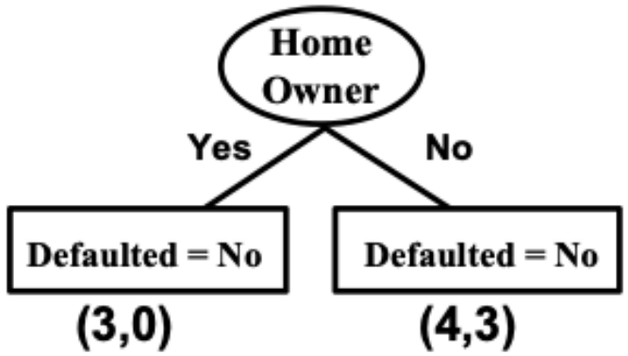


ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Decision Tree Induction (Hunt's Algorithm)



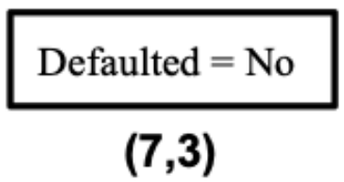
(a)



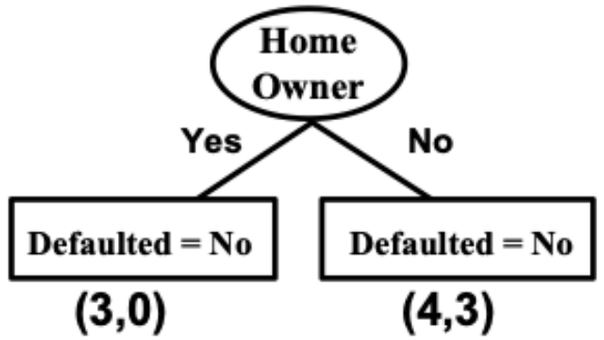
(b)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Decision Tree Induction (Hunt's Algorithm)

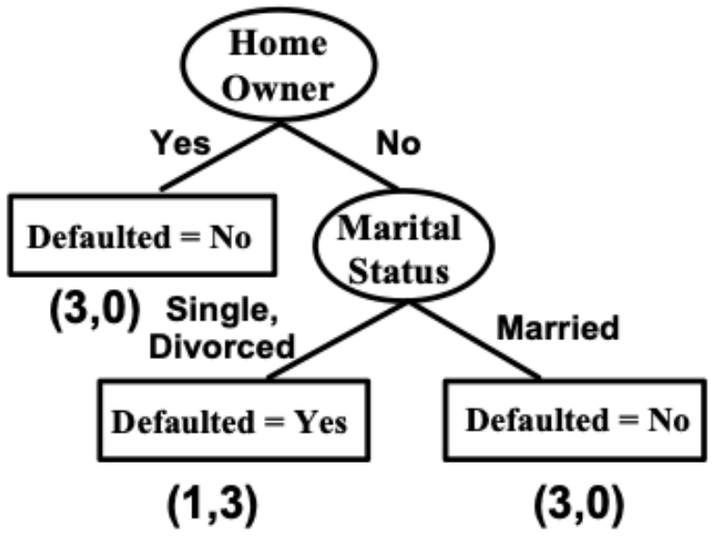


(a)



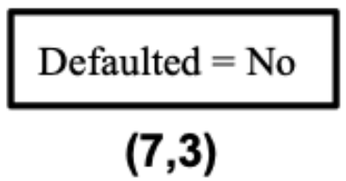
(b)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

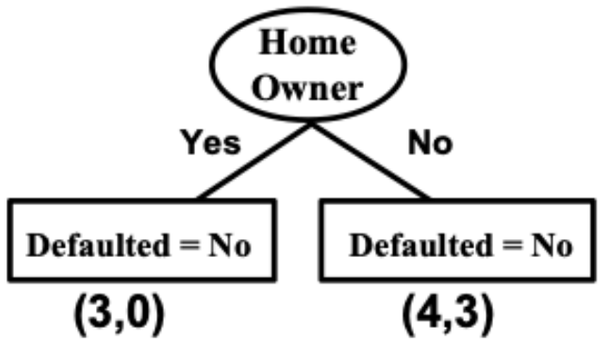


(c)

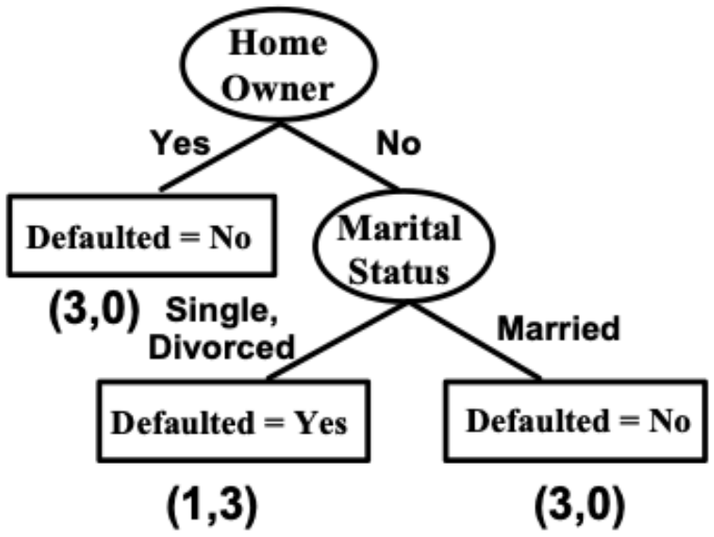
Decision Tree Induction (Hunt's Algorithm)



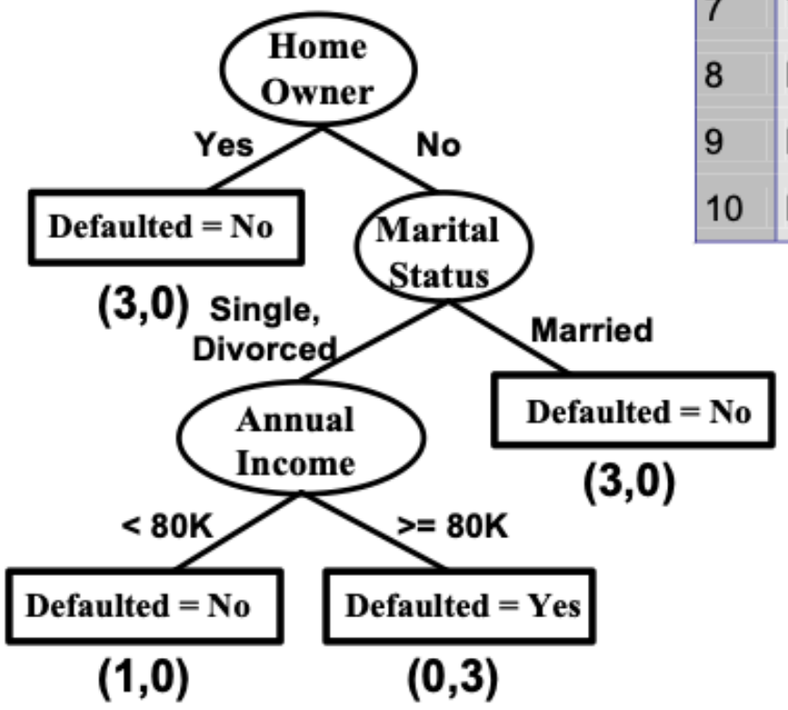
(a)



(b)



(c)



(d)

ID	Home Owner	Marital Status	Annual Income	Defaulted Borrower
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Design Issues

- How should training records be split?
- How should the splitting procedure stop?

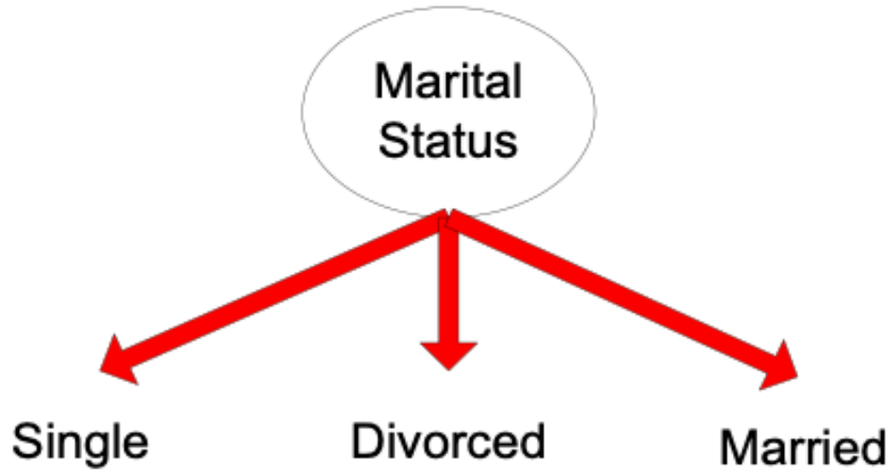
Methods for Expressing Test Conditions

Depends on attribute types

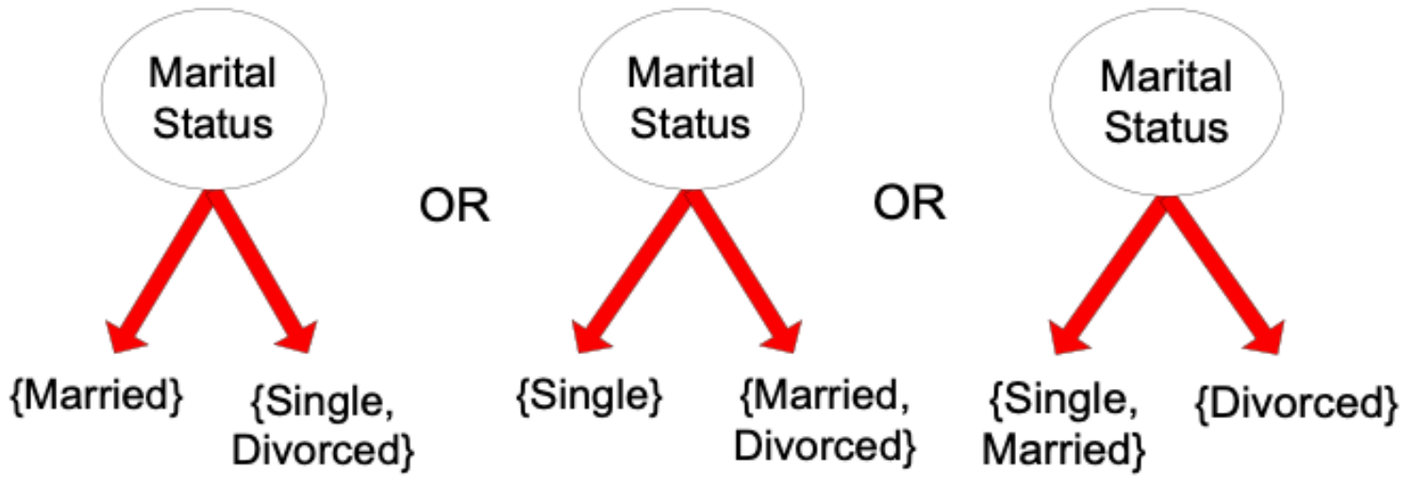
- Binary
- Nominal: A type of categorical variable used to label or name data into distinct, non-numerical groups that have no inherent, natural order or ranking.
- Ordinal: A type of categorical variable with categories that have a meaningful logical order or ranking, but the distances between the categories aren't necessarily equal or known, unlike quantitative variables. You can say one category is "more than" or "less than" another, but not *how much* more or less. Common examples include education levels ("high school" to "PhD"), satisfactory ratings ("low" to "high")
- Continuous: A type of quantitative (numerical) variable that can take on an infinite number of values within a given range, including decimals and fractions. Unlike discrete variables, which are counted, continuous variables are measured (time, weight, height, temperature).

Test Condition for Nominal Attributes

- Multi-way split:
  - Use as many partitions as distinct values.

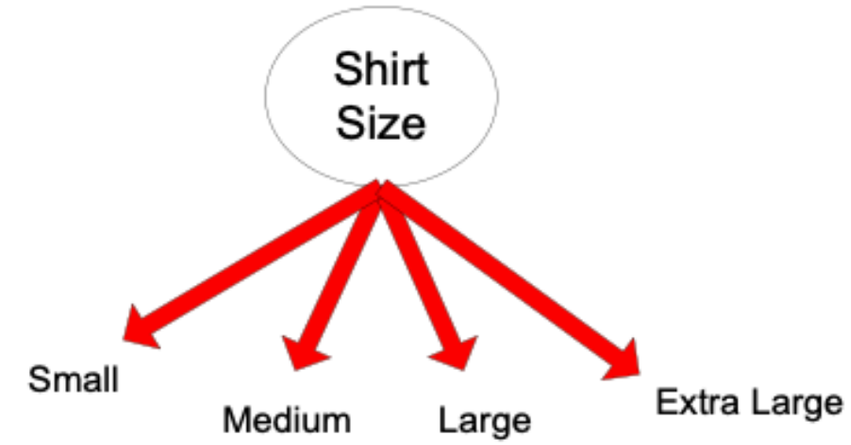


- Binary split:
  - Divides values into two subsets

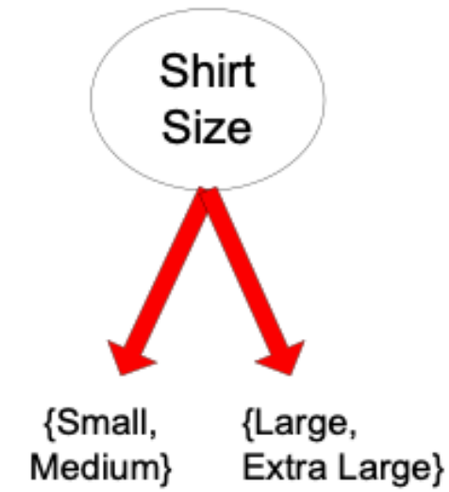


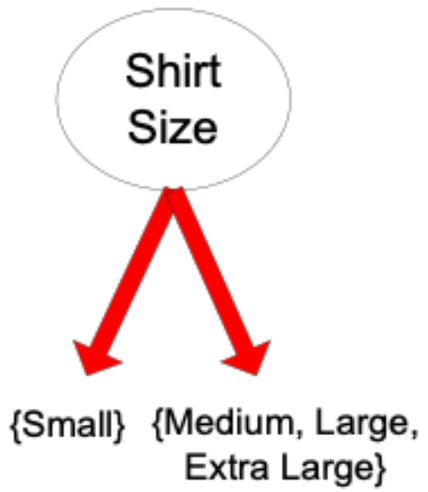
Test Condition for Ordinal Attributes

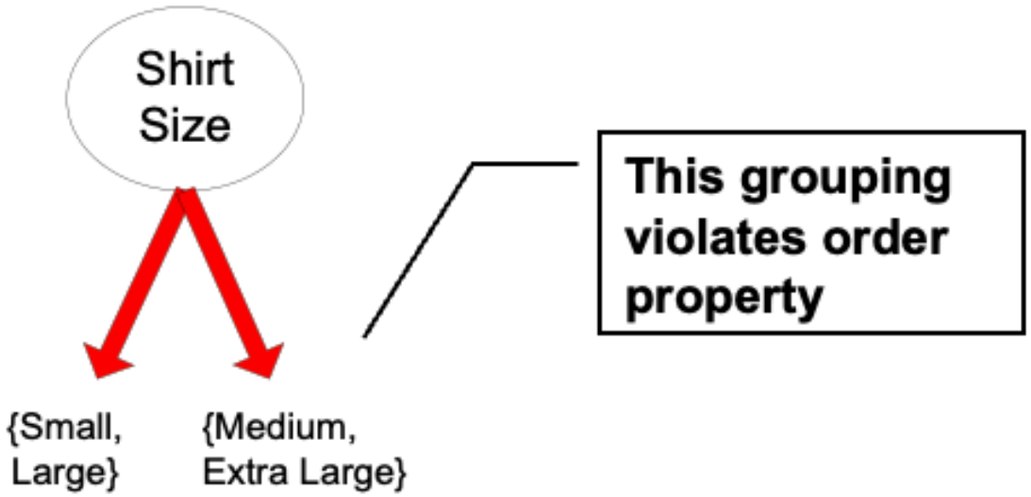
- Multi-way split:
  - Use as many partitions as distinct values



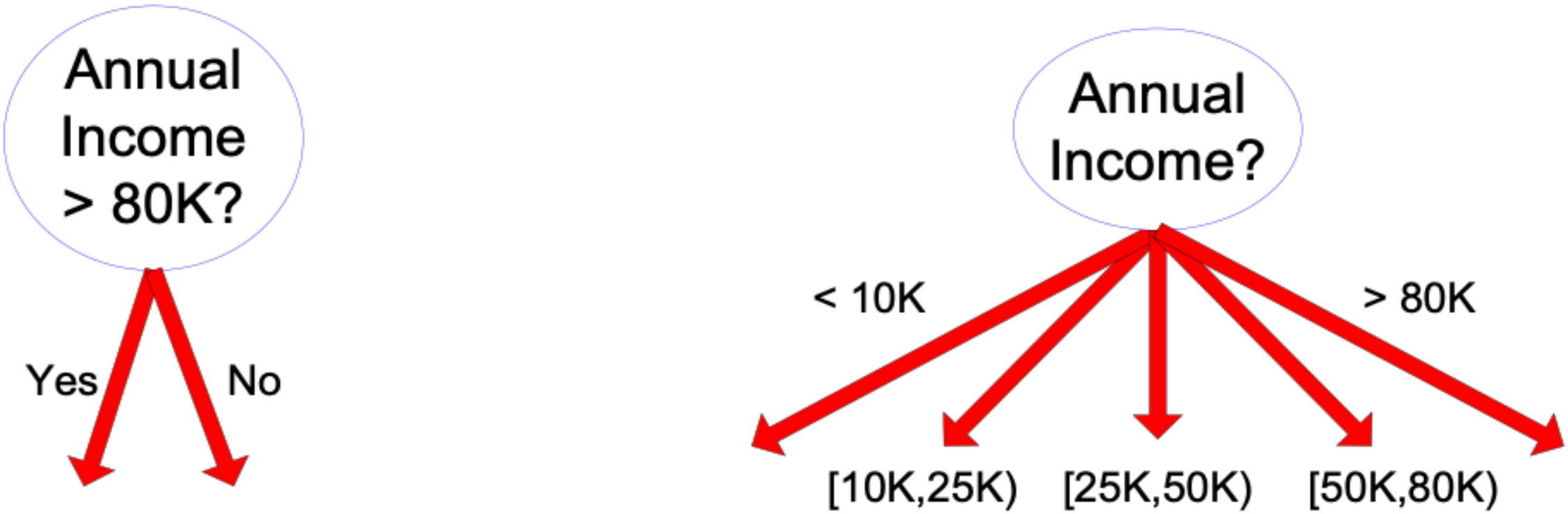
- Binary split:
  - Divides values into two subsets
  - Preserve order property among attribute values







Test Condition for Continuous Attributes



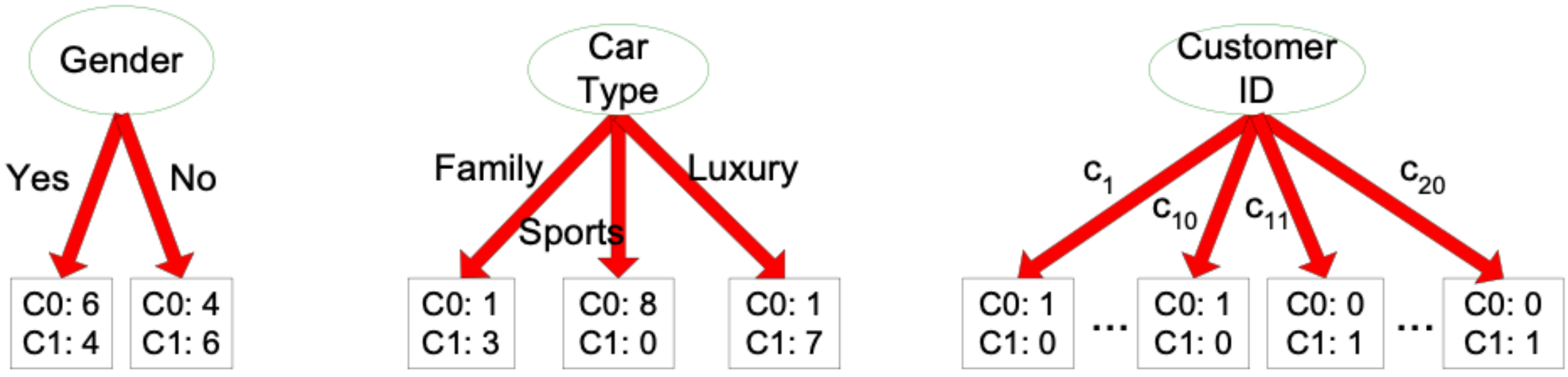
(i) Binary split

(ii) Multi-way split

How to Determine the Best Split?

Before Splitting: 10 records of class 0,
10 records of class 1

Customer Id	Gender	Car Type	Shirt Size	Class
1	M	Family	Small	C0
2	M	Sports	Medium	C0
3	M	Sports	Medium	C0
4	M	Sports	Large	C0
5	M	Sports	Extra Large	C0
6	M	Sports	Extra Large	C0
7	F	Sports	Small	C0
8	F	Sports	Small	C0
9	F	Sports	Medium	C0
10	F	Luxury	Large	C0
11	M	Family	Large	C1
12	M	Family	Extra Large	C1
13	M	Family	Medium	C1
14	M	Luxury	Extra Large	C1
15	F	Luxury	Small	C1
16	F	Luxury	Small	C1
17	F	Luxury	Medium	C1
18	F	Luxury	Medium	C1
19	F	Luxury	Medium	C1
20	F	Luxury	Large	C1



Which test condition is the best?

How to Determine the Best Split?

- Greedy approach:
  - Nodes with purer class distribution are preferred

- Need a measure of node impurity:

C0: 5 C1: 5

High degree of impurity

C0: 9 C1: 1

Low degree of impurity

Measures of Node Impurity

- Impurity measures for a single node

$$\text{Gini index} = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

$$\text{Entropy} = -\sum_{I=0}^{c-1} p_i(t) \log_2 p_i(t)$$

$$\text{Classification error} = 1 - \max_i [p_i(t)]$$

where $p_i(t)$ is the relative frequency of training instances that belong to class I at node t, c is the total number of classes, and $0 \log_2 0 = 0$ in entropy calculations.

Finding the Best Split

- Compute impurity measure P before splitting
- Compute impurity measure M after splitting
  - Compute impurity measure of each child node
  - M is the weighted impurity of child nodes
- Choose the attribute test condition that produces the highest gain

$$\text{Gain} = P - M$$

or equivalently lowest impurity measure M after splitting.

Finding the Best Split

Before Splitting:

C0	**N00**
C1	**N01**

→ **P**



C0	**N10**
C1	**N11**

C0	**N20**
C1	**N21**

C0	**N30**
C1	**N31**

C0	**N40**
C1	**N41**

M11 **M12** **M21** **M22**

M1 **M2**

Gain = P – M1 vs P – M2

Measures of Node Impurity: GINI

Gini index for a given node t:

$$\text{Gini index} = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Properties

- Maximum of $1 - \frac{1}{c}$ when records are equally distributed among all classes, implying the least beneficial situation for classification
- Minimum of 0 when all records belong to one class, implying the most beneficial situation for classification

Gini Index for a Collection of Nodes

- When a node p is split into k partitions (children), compute the weighted impurity for all the child nodes:

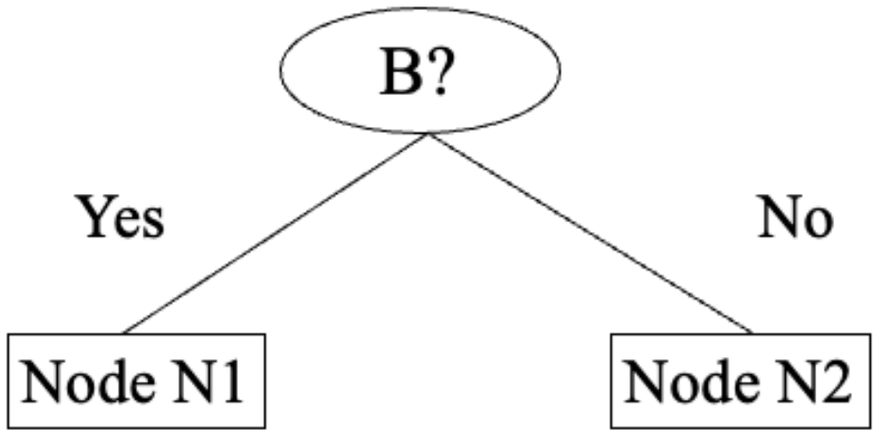
$$GINI_{split} = \sum_{i=1}^{k} \frac{n_i}{n} \text{GINI}(i)$$

where:

- n_i = number of records at child i.
- n = number of records at parent node p.

Gini Index for a Collection of Nodes

- Splits into two partitions (child nodes)
- Effect of Weighing partitions:
  - Larger and purer partitions are sought



	Parent
C1	**7**
C2	**5**
Gini = 0.486	

Gini(N1)
= 1 – (5/6)² – (1/6)²
= 0.278

Gini(N2)
= 1 – (2/6)² – (4/6)²
= 0.444

	N1	N2
C1	**5**	**2**
C2	**1**	**4**
Gini=0.361		

Weighted Gini of N1 N2
= 6/12 * 0.278 +
6/12 * 0.444
= 0.361

Gain = 0.486 – 0.361 = 0.125

Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
Annual Income										
Sorted Values →	60	70	75	85	90	95	100	120	125	220

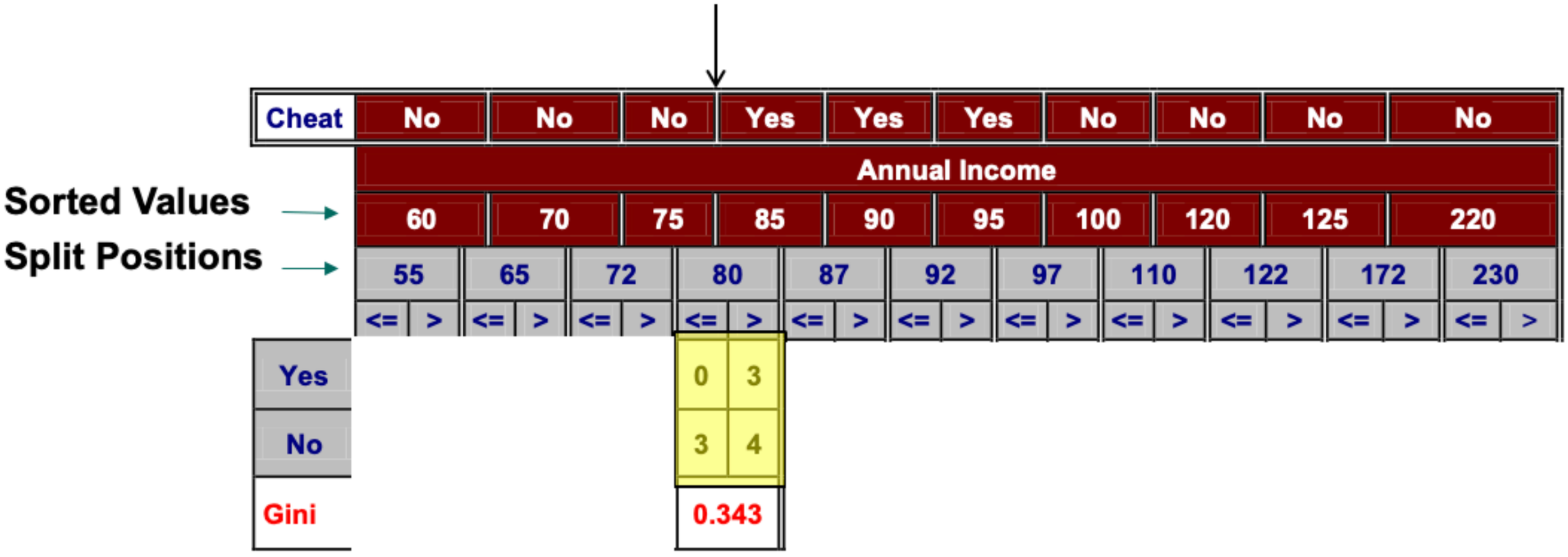
Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
Annual Income										
Sorted Values →	60	70	75	85	90	95	100	120	125	220
Split Positions →	55	65	72	80	87	92	97	110	122	172	230
	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >

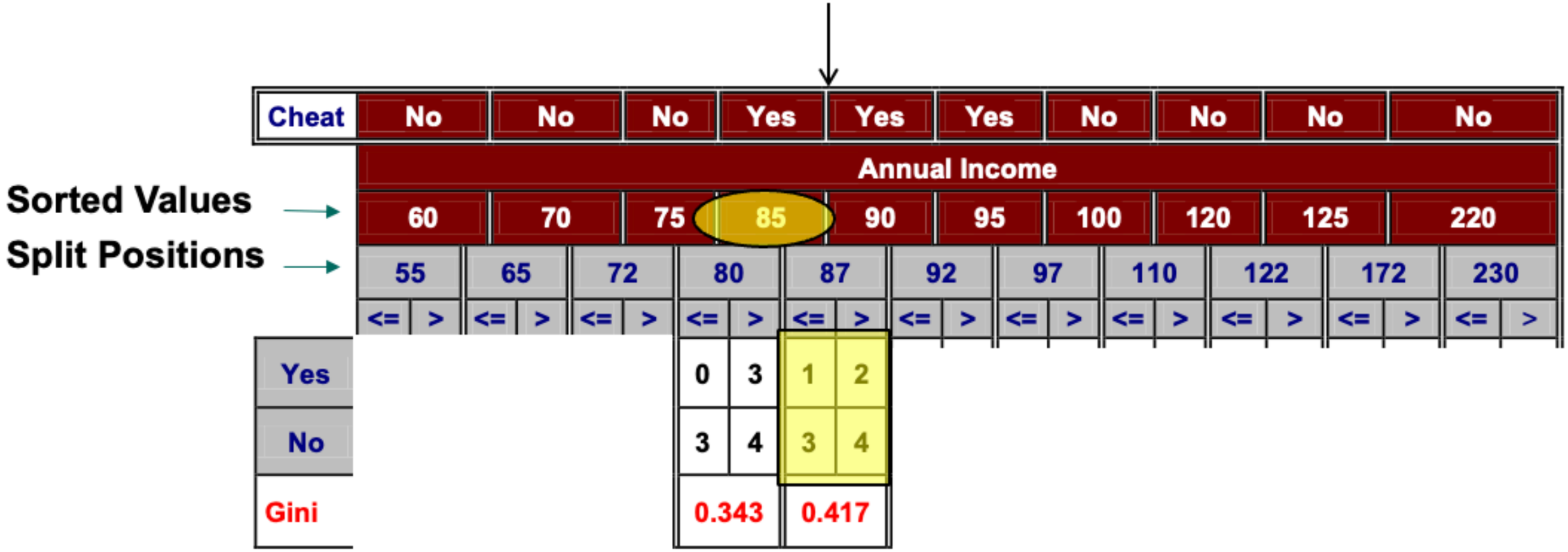
Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index



Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index



Continuous Attributes

- For efficient computation: for each attribute,
  - Sort the attribute on values
  - Linearly scan these values, each time updating the count matrix and computing gini index
  - Choose the split position that has the least gini index

Cheat	No	No	No	Yes	Yes	Yes	No	No	No	No
Annual Income										
Sorted Values →	60	70	75	85	90	95	100	120	125	220
Split Positions →	55	65	72	80	87	92	97	110	122	172	230
	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >	<= >
Yes	0 3	0 3	0 3	0 3	1 2	2 1	3 0	3 0	3 0	3 0	3 0
No	0 7	1 6	2 5	3 4	3 4	3 4	3 4	4 3	5 2	6 1	7 0
Gini	0.420	0.400	0.375	0.343	0.417	0.400	***0.300***	0.343	0.375	0.400	0.420

Measures of Node Impurity: Entropy

- Entropy at a given node t:

$$\text{Entropy} = -\sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

where $p_i(t)$ is the frequency of class i at node t, and c is the total number of classes.

- Properties
  - Maximum of $\log_2 c$ when records are equally distributed among all classes, implying the least beneficial situation for classification.
  - Minimum of 0 when all records belong to one class, implying most beneficial situation for classification.

Entropy of a Single Node

$$Entropy = -\sum_{i=0}^{c-1} p_i(t) log_2 p_i(t)$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Entropy = – 0 log 0 – 1 log 1 = – 0 – 0 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Entropy = – (1/6) $\log_2$ (1/6) – (5/6) $\log_2$ (1/6) = 0.65

C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

Entropy = – (2/6) $\log_2$ (2/6) – (4/6) $\log_2$ (4/6) = 0.92

Information Gain after Splitting

- Information Gain:

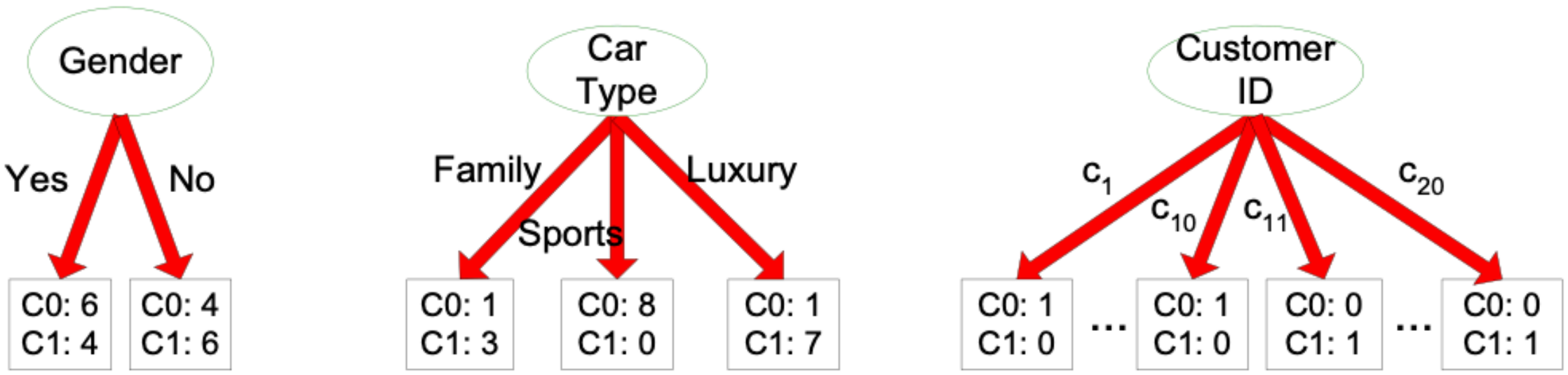
$$\text{Gain}_{split} = \text{Entropy}(p) - \sum_{i=1}^{k} \frac{n_i}{n} \text{Entropy}(i)$$

where the parent node p, is split into k partitions (children). n_i is the number of records in child node i.

- Choose the split that achieves the most information gain.

Problem with Large Number of Partitions

- Node impurity measures tend to prefer splits that result in large number of partitions, each being small but pure



  - Customer ID has highest information gain because entropy for all the children is zero

Gain Ratio

- Gain Ratio:

$$\text{Gain Ratio} = \frac{\text{Gain}_{\text{split}}}{\text{Split Info}}$$

where

$$\text{Split Info} = -\sum_{i=1}^{k} \frac{n_i}{n} \log_2 \frac{n_i}{n}$$

Parent node is split into k partitions (children). n_i is the number of records in child node i.

- Designed to overcome the disadvantage of Information Gain.

	CarType		
	Family	Sports	Luxury
C1	1	8	1
C2	3	0	7
Gini	0.163		

SplitINFO = 1.52

	CarType	
	{Sports, Luxury}	{Family}
C1	9	1
C2	7	3
Gini	0.468	

SplitINFO = 0.72

	CarType	
	{Sports}	{Family, Luxury}
C1	8	2
C2	0	10
Gini	0.167	

SplitINFO = 0.97

Measures of Impurity: Classification Error

- Classification error at a node t:

$$\text{Classification Error}(t) = 1 - \max_i [p_i(t)]$$

- Properties
  - Maximum of $1 - \frac{1}{c}$ when records are equally distributed among all classes, implying the least interesting situation
  - Minimum of 0 when all records belong to one class, implying the most interesting situation

Measures of Impurity: Classification

$$Error(t) = 1 - \max_i [p_i(t)]$$

C1	**0**
C2	**6**

P(C1) = 0/6 = 0 P(C2) = 6/6 = 1

Error = 1 – max (0, 1) = 1 – 1 = 0

C1	**1**
C2	**5**

P(C1) = 1/6 P(C2) = 5/6

Error = 1 – max (1/6, 5/6) = 1 – 5/6 = 1/6

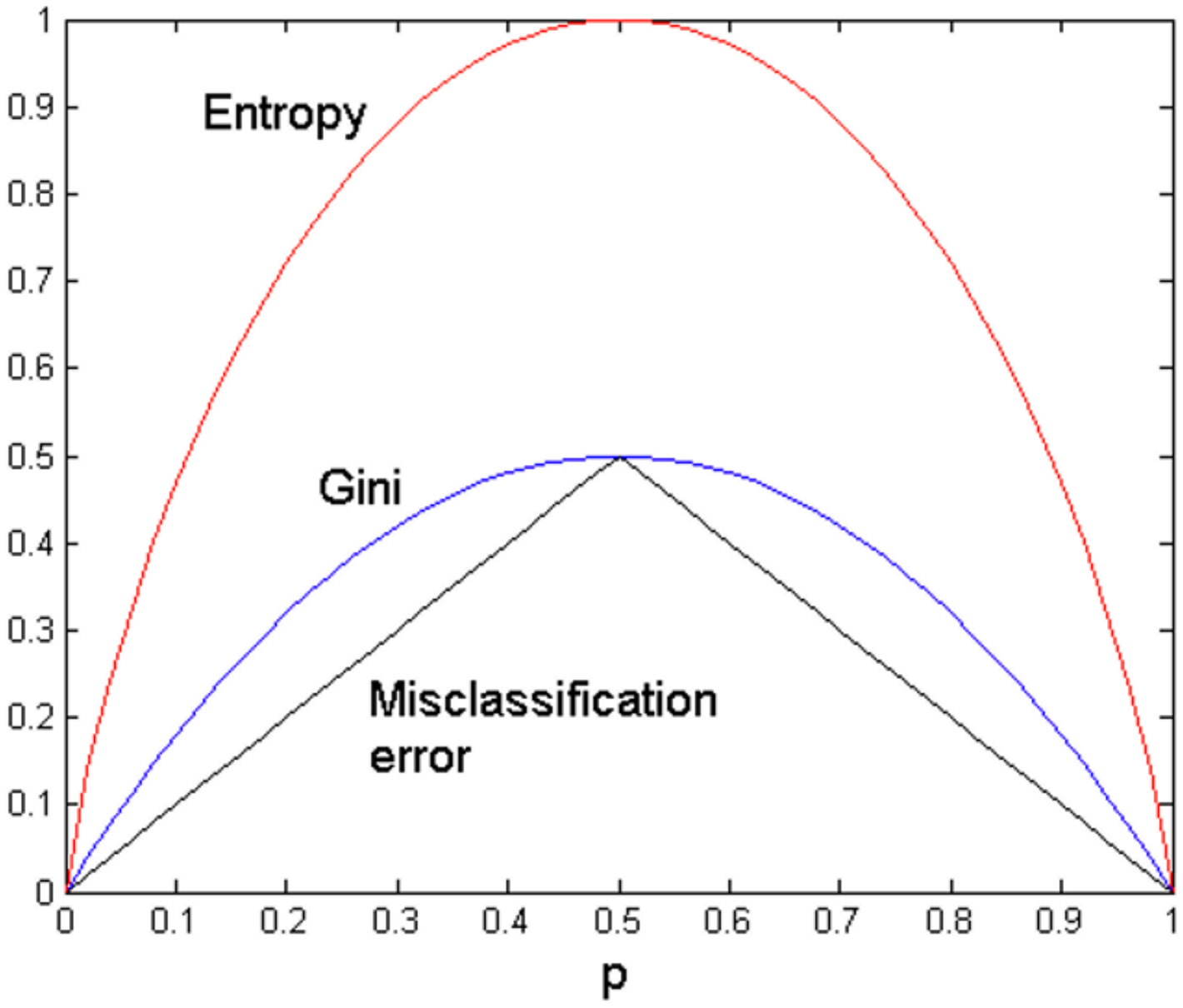
C1	**2**
C2	**4**

P(C1) = 2/6 P(C2) = 4/6

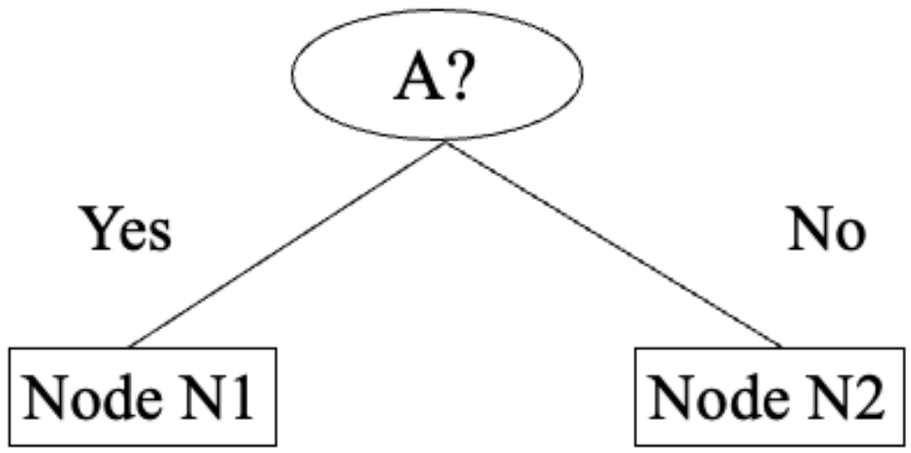
Error = 1 – max (2/6, 4/6) = 1 – 4/6 = 1/3

Comparison among Impurity Measures

For a 2-class problem:



Classification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

Gini(N1)
= 1 – (3/3)² – (0/3)²
= 0

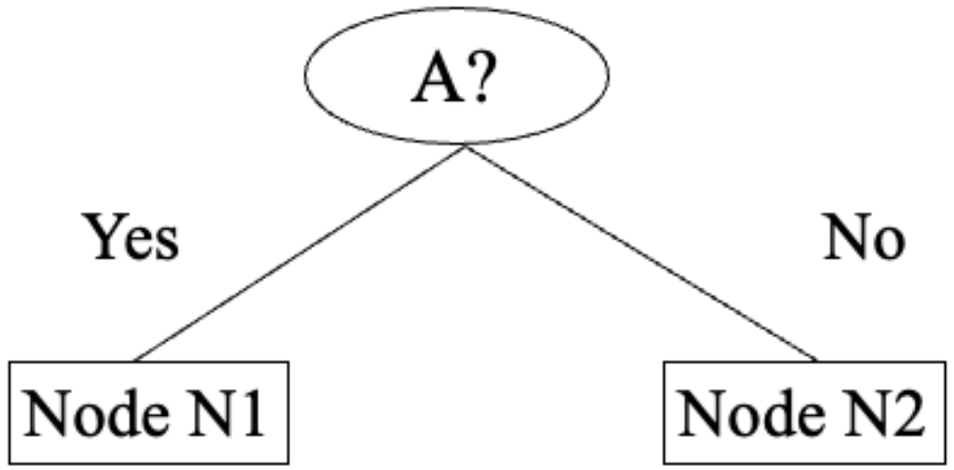
Gini(N2)
= 1 – (4/7)² – (3/7)²
= 0.489

	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

Gini(Children)
= 3/10 * 0
+ 7/10 * 0.489
= 0.342

Gini improves but error remains the same!!

Classification Error vs Gini Index



	Parent
C1	7
C2	3
Gini = 0.42	

	N1	N2
C1	3	4
C2	0	3
Gini=0.342		

	N1	N2
C1	3	4
C2	1	2
Gini=0.416		

Misclassification error for all three cases = 0.3 !

Advantages

- Simple to understand and to interpret. Trees can be visualized.
- Requires little data preparation. Other techniques often require data normalization, dummy variables need to be created and blank values to be removed.
- Able to handle both numerical and categorical data. Other techniques are usually specialized in analyzing datasets that have only one type of variable.
- Able to handle multi-output problems.
- Uses a white box model. If a given situation is observable in a model, the explanation for the condition is easily explained by boolean logic. By contrast, in a black box model (e.g., in an artificial neural network), results may be more difficult to interpret.

https://scikit-learn.org/stable/modules/tree.html

Disadvantages

- Decision-tree learners can create over-complex trees that do not generalize the data well. This is called overfitting. Mechanisms such as pruning (not currently supported), setting the minimum number of samples required at a leaf node or setting the maximum depth of the tree are necessary to avoid this problem.

- Decision trees can be unstable because small variations in the data might result in a completely different tree being generated. This problem is mitigated by using decision trees within an ensemble.

- The problem of learning an optimal decision tree is known to be NP-complete under several aspects of optimality and even for simple concepts. Consequently, practical decision-tree learning algorithms are based on heuristic algorithms such as the greedy algorithm where locally optimal decisions are made at each node. Such algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees in an ensemble learner, where the features and samples are randomly sampled with replacement.

https://scikit-learn.org/stable/modules/tree.html

Disadvantages

- There are concepts that are hard to learn because decision trees do not express them easily, such as XOR, parity or multiplexer problems.
- Decision tree learners create biased trees if some classes dominate. It is therefore recommended to balance the dataset prior to fitting with the decision tree.

https://scikit-learn.org/stable/modules/tree.html

Decision Tree Core Idea

A decision tree:

- Recursively splits the feature space
- Uses if–then rules
- Predicts by following a path from root to leaf

Decision Trees for Regression

Decision Tree for Classification

- The outcome is categorical (e.g., spam vs not spam, disease vs no disease)
- Leaf prediction: Class label (or class probabilities)
- Split criteria: Gini index, entropy, or classification error
- Typical uses
  - Medical diagnosis
  - Credit approval
  - Species classification (Iris data)

Decision Tree for Regression

- The outcome is continuous (e.g., house price, income, concentration)
- Leaf prediction: A number (usually the mean of samples in the leaf)
- Split criteria: Variance reduction, Mean Squared Error (MSE)
- Typical uses
  - Price prediction
  - Rick scores
  - Environmental or exposure levels

Decision Tree for Regression

A regression tree chooses splits that make the target values as similar as possible within each leaf.

In other words: Good splits create groups where the numbers don't vary much.

Decision Tree for Regression

Step 1: What does a leaf predict?

In a regression tree, each leaf predicts:

$$\hat{y}_{\text{leaf}} = \frac{1}{n_{\text{leaf}}} \sum_{i \in \text{leaf}} y_i$$

So:

- The prediction is the mean of the target values in that leaf.
- This choice minimizes squared error.

Decision Tree for Regression

Step 2: Error inside a leaf (variance/MSE)

For a node (or leaf) with values $y_1, y_2, \cdots, y_n$:

- Variance

$$\text{Var} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y}_i)^2$$

- Mean Squared Error (same idea):

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Since $\hat{y} = \bar{y}$, variance and MSE are equivalent here.

Key point: Regression trees minimize within-node squared error.

Decision Tree for Regression

Step 3: Error before the split (Parent)

Suppose a node contains n samples.

Compute:

- Mean: $\bar{y}_{\text{parent}}$
- Variance:

$$Var_{\text{parent}} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \bar{y}_{\text{parent}})^2$$

This is how "messy" the node is before splitting.

Decision Tree for Regression

Step 4: Error after the split (Children)

A candidate split divides the data into:

- Left child L with n_L samples
- Right child R with n_R samples

Compute:

$$\text{Var}_L = \frac{1}{n_L} \sum_{i \in L} (y_i - \bar{y}_L)^2 \tag{1}$$

$$\text{Var}_R = \frac{1}{n_L} \sum_{i \in R} (y_i - \bar{y}_R)^2 \tag{2}$$

Then take the weighted average:

$$\text{weighted Var}_{\text{children}} = \frac{n_L}{n} \text{Var}_L + \frac{n_R}{n} \text{Var}_R$$

Decision Tree for Regression

Step 5: Split criterion (Variance reduction)

The quality of a split is measured by how much variance it removes:

$$\text{Variance Reduction} = \text{Var}_{\text{parent}} - \left(\frac{n_L}{n} \text{Var}_L + \frac{n_L}{n} \text{Var}_R \right)$$

Decision rule: Choose the split that maximizes variance reduction (equivalently, minimizes weighted child variance).

Decision Tree for Regression: Example

Suppose parent node has:

$$y = [2, 3, 8, 9]$$

Parent mean: $\bar{y} = 5.5$

Try split:

- Left: [2, 3], mean=2.5, variance = 0.25
- Right: [8, 9], mean = 8.5, variance = 0.25

Weighted child variance:

$$\frac{2}{4}(0.25) + \frac{2}{4}(0.25) = 0.25$$

Parent variance:

$$\frac{1}{4}\left[(2 - 5.5)^2 + \cdots\right] = 9.25$$

Variance reduction:

$$9.25 - 0.25 = 9.0 (\text{excellent split})$$

Decision Trees Overfitting

Overfitting

- Overfitting means that the model:
  - Fits training date extremely well.
  - Learns noise instead of structures in the data
  - Performs poorly on new (test) data.
- Formal definition:

$$\text{Error}_{\text{train}} \ll \text{Error}_{\text{test}}$$

  - Large $\text{Error}_{\text{test}} - \text{Error}_{\text{train}}$ means overfitting
  - Small $\text{Error}_{\text{test}} - \text{Error}_{\text{train}}$ means better generalization.

Bias-Variance Tradeoff

- Bias
  - Systematic error from model assumptions
  - Difference between the average model prediction and the true function

$$\text{Bias}(x) = E\left[\hat{f}(x)\right] - f(x)$$

- Variance
  - Sensitivity of the model to the training data
  - Variability of predictions across different training sets

$$\text{Var}(x) = E\left[\left(\hat{f}(x) - E\left[\hat{f}(x)\right]\right)^2\right]$$

- Key relationship

$$E\left[\left(Y - \hat{f}(X)\right)^2\right] = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Bias-Variance Tradeoff

- Bias
  - Bias measures how far the average learned model is from the truth
  - Bias is error from oversimplification.
- Variance
  - Variance measures how much the model changes if we retrain it on new data
  - Variance is error from sensitivity to data.
- Expectations E are over different training datasets, not over samples within one dataset.
- Overfitting corresponds to variance dominating bias
- Decision trees
  - Shallow trees $\rightarrow$ high bias, low variance
  - Deep trees $\rightarrow$ low bias, high variance

Decision Tree

Increasing model complexity will cause bias to decrease and the variance to increase, leading to overfitting.

Decision Tree Complexity

In practice, Decision Trees complexity is defined using structural properties of the tree, including the following:

- Tree Depth (most common proxy)

$$\text{Large Complexity} \iff \text{Large Depth}$$

  - Depth here is defined as the number of splits along the longest root-to-leaf path.
  - Deeper trees result in lower bias and higher variance.
  - In scikit-learn: `DecisionTreeClassifier(max_depth=k)`

Decision Tree Complexity

- Number of leaves (model capacity measure)

$$\text{Complexity} \propto \text{Number of leaves}$$

  - Each leaf corresponds to a distinct prediction region
  - More leaves $\rightarrow$ more piecewise constant regions

- Number of internal nodes/splits. Each internal node adds one decision rule.

- Effective complexity. In cost-complexity pruning, complexity is explicitly defined as:

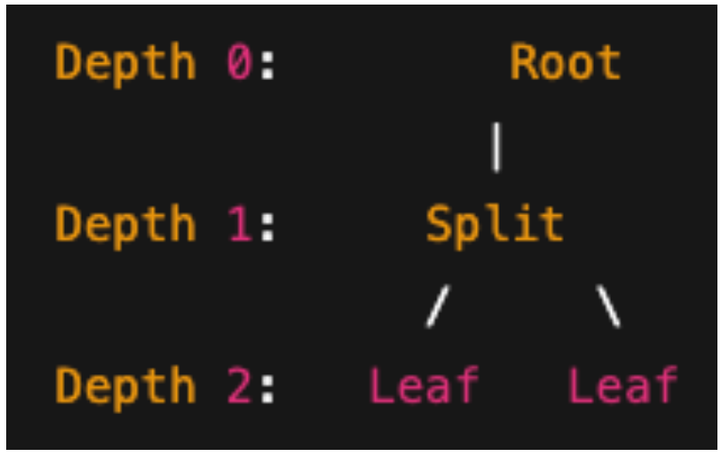
$$\text{Loss} = \text{Training Error} + \alpha \times |\text{tree}|$$

  where $|\text{tree}|$ =number of terminal nodes (leaves) and α controls how much complexity is penalized. This is also the most formal definition used in tree theory.

Decision Tree Depth

Conventino used in ML, including scikit-learn

- Root node is at depth 0.
- Each split increases depth by 1.



Relation to overfitting:

- Large depth $\Longrightarrow$ more complexity
- Large depth $\Longrightarrow$ more variance
- Very deep trees $\Longrightarrow$ overfitting

This is why we control depth in scikit-learn using:

```
DecisionTreeClassifier(max_depth=k)
```

Decision Tree Overfitting

Why trees overfit easily?

- Greedy splitting
- Can keep splitting until:
  - Every leaf has 1 sample.
  - Training error = 0
  - But Test error could be very high.
  - The tree has memorized the dataset.

Depth	Training Error	Test Error
Low	High	High
Medium	Medium	Lowest
High	Very low	High

Diagnosing Overfitting

- **Train/Test split:** Compare training accuracy with test accuracy. If large gap between the two, then overfitting has occurred.
- **Cross-validation:** Estimate generalization error more reliably.

How to Prevent Overfitting

- Early stopping: Do NOT let the tree grow too much.
  - **Train/Test split:** Compare training accuracy with test accuracy. If large gap between the two, then overfitting has occurred.
  - **Cross-validation:** Estimate generalization error more reliably.
- Post-Pruning (Cost-Complexity Pruning) by optimize

$$\text{Loss} = \text{Training Error} + \alpha \times \text{Tree Size}$$

where α penalizes complexity and larger $\alpha \rightarrow$ smaller tree.

- Ensembles: Random forests reduce variance by averaging.